

Package ‘PDRobust’

October 2, 2026

Title Robust Longitudinal Effects Under Truncation by Death

Version 0.3.8

Description Implements principal-stratification methods for estimating time-specific and pooled heterogeneous treatment effects in longitudinal studies where outcomes may be truncated by death. Supports continuous and binary outcomes, explicit data validation and standardization, and covariate-dependent treatment effects. Fits propensity-score, principal-score, and outcome models and provides subject-level bootstrap inference, covariate-balance diagnostics, principal-stratum summaries, treatment-group-specific survival odds ratios, and outcome-noise sensitivity analysis. Methodological background is provided in [doi:10.48550/arXiv.2608.06654](https://doi.org/10.48550/arXiv.2608.06654).

License MIT + file LICENSE

Encoding UTF-8

Depends R (>= 4.2.0)

Imports ggplot2, quantreg, rootSolve, stats, utils

Suggests knitr, ragg, rmarkdown, testthat (>= 3.0.0)

Config/testthat/edition 3

Config/Needs/coverage covr

Config/Needs/website pkgdown

VignetteBuilder knitr

LazyData true

LazyDataCompression xz

URL https://github.com/whhuan/PD_Robust,
https://whhuan.github.io/PD_Robust/

BugReports https://github.com/whhuan/PD_Robust/issues

Config/roxygen2/version 8.0.0

NeedsCompilation no

Author Huan Wang [aut, cre],
Yilin Zhang [aut]

Maintainer Huan Wang <whhuan42@gmail.com>
Repository CRAN
Date/Publication 2026-10-02 11:20:02 UTC

Contents

BiSample	2
DataCheck	3
DataStandard	4
HTEAllT	5
HTESepT	8
ImperfectConSample	10
Mapping	11
ORCI	12
OutPred	13
PrinPred	14
PrinSDiag	15
print.pd_mapping	16
PSDiag	18
PSPred	19
QR	20
SA	21
Index	23

BiSample	<i>Binary longitudinal example data</i>
----------	---

Description

Simulated long-format data for illustrating binary-outcome analyses. Potential survival is generated with $S^1 \geq S^0$, matching the package’s treatment-1 survival-favorable convention. The simulation variables are not observed counterfactual information available in a real study.

Usage

BiSample

Format

- A simulated long-format data frame with 1,200 rows (400 subjects at three visits) and 16 variables:
- id** Subject identifier.
 - time** Analysis time.
 - Pi** Simulated probability of treatment 1 conditional on baseline covariates, stored to three decimal places.

S1, S0 Simulated potential survival indicators under treatment 1 and 0, respectively; 1 denotes alive and 0 denotes dead.

Y1, Y0 Simulated binary potential outcomes under treatment 1 and 0, respectively. These simulation variables are retained for illustration; package analyses use the observed outcome *Y*.

X1, X2, X3 Continuous baseline covariates.

X4, X5, X6 Binary baseline covariates.

A Binary treatment indicator.

S Binary survival or intermediate-status indicator.

Y Binary outcome, structurally missing after death.

Source

Simulated for package examples.

Examples

```
data("BiSample", package = "PDRobust")
head(BiSample)
```

DataCheck

Check whether longitudinal data are ready for analysis

Description

Checks the columns, values, visit structure, and analysis settings specified by mapping. Every observed time from baseline through the cutoff is treated as an analysis time, and the input data are left unchanged.

Usage

```
DataCheck(data, mapping, strict = FALSE)
```

Arguments

<code>data</code>	A long-format data frame.
<code>mapping</code>	A <code>pd_mapping</code> object returned by <code>Mapping()</code> .
<code>strict</code>	If TRUE, stop as soon as a problem that prevents analysis is found. If FALSE, return a report describing all checks that can be completed.

Value

A `pd_data_check` list with the following components:

valid TRUE when no check classified as an error fails. Some warnings about encoding or ordering may still prevent analysis.

ready_for_analysis TRUE when the data pass every check required for analysis.

manual_resolution_required TRUE when a failed check requires the user to correct the data before standardization.

can_standardize TRUE when no problem requires manual correction. Standardization can still fail if rows must be removed but `drop = FALSE`, or if removal leaves no observations or only one treatment group.

checks A data frame with one row per performed check, including the result, its importance, details, and a recommended action.

settings A list containing the validated mapping.

diagnostics Detailed row indices, subject identifiers, and summary tables for the performed checks. Missing columns or empty input cause an early return with only the checks possible at that stage.

Numeric summaries intended for display are rounded to three decimals; counts, row indices, identifiers, and logical flags retain their types.

Examples

```
data("BiSample", package = "PDRobust")
map <- Mapping(
  id = "id", time = "time", treatment = "A",
  survival = "S", outcome = "Y",
  baseline_time = 0, cutoff_time = 2,
  covariates = c("X1", "X2", "X4"),
  interest_vars = c("X1", "X2"), y_type = "B"
)
check <- DataCheck(BiSample, map)
check$ready_for_analysis
```

DataStandard

Prepare longitudinal data for PDRobust analyses

Description

Converts supported binary values to 0 and 1, replaces subject IDs and analysis times with consecutive integers, sorts the data by subject and time, and stores the information needed by other PDRobust functions.

Usage

```
DataStandard(data, mapping, drop = FALSE)
```

Arguments

<code>data</code>	A long-format data frame.
<code>mapping</code>	A <code>pd_mapping</code> object returned by <code>Mapping()</code> .
<code>drop</code>	If <code>TRUE</code> , remove rows that cannot be assigned to a subject and remove subjects with missing visits or required values between baseline and cutoff. The returned report records what was removed. If <code>FALSE</code> , these problems stop standardization.

Value

A data frame inheriting from `pd_data`, retaining the input column names and additional unmapped columns. Rows outside the mapped time window are removed; retained rows are sorted by recoded ID and time. Attributes are:

`pd_mapping` The mapping updated to use the standardized baseline and cutoff times. Column names remain unchanged.

`pd_original_mapping` The mapping supplied for the raw data.

`pd_check` The final `pd_data_check` report, including attrition. A warning is issued if the returned data are not ready for analysis, for example if deletion removes one treatment group.

`pd_standardization` A list containing `time_map`, `id_map`, `attrition`, and `initial_check`. The maps link raw values to their standardized values; the initial check describes the input data.

The function checks the data both before and after preparation. The stored reports describe this call and are not recalculated if the returned data are later edited or subsetted; see [pd_methods](#). The ID map has one row per retained subject, so its size increases with the number of subjects. Analysis values retain full precision; only summaries shown to users and percentages describing removed data are rounded.

Examples

```
data("BiSample", package = "PDRobust")
map <- Mapping(
  id = "id", time = "time", treatment = "A",
  survival = "S", outcome = "Y",
  baseline_time = 0, cutoff_time = 2,
  covariates = c("X1", "X2", "X4"),
  interest_vars = c("X1", "X2"), y_type = "B"
)
pd_dat <- DataStandard(BiSample, map)
attr(pd_dat, "pd_mapping")
```

Description

Performs joint longitudinal analysis of heterogeneous treatment effects across all time points with time included as a covariate.

Usage

```
HTEAllT(
  data,
  ps_fo,
  prin_fo,
  out_fo,
  B,
  conf_level = 0.95,
  max_attempts = NULL,
  verbose = TRUE,
  progress_callback = NULL
)
```

Arguments

<code>data</code>	Data prepared by <code>DataStandard()</code> .
<code>ps_fo</code>	propensity score model formula
<code>prin_fo</code>	principal score model formula
<code>out_fo</code>	outcome mean model formula
<code>B</code>	Number of bootstrap replications. Use 0 for point estimates only.
<code>conf_level</code>	The confidence level for Wald intervals calculated from bootstrap standard errors.
<code>max_attempts</code>	The maximum number of resampling attempts allowed to obtain B successful bootstrap replications. Defaults to 10B. Resampling stops once B successful replications are obtained or when the maximum number of attempts is reached, whichever occurs first. Thus, fewer than B successful replications may be returned if the maximum number of attempts is reached.
<code>verbose</code>	If TRUE, print bootstrap progress messages.
<code>progress_callback</code>	An optional function for receiving bootstrap progress updates. It is called before model fitting, after the point estimate, after every bootstrap attempt, and when bootstrapping finishes. Each update is a named list containing <code>stage</code> , <code>successful</code> , <code>requested</code> , <code>attempts</code> , <code>max_attempts</code> , <code>failed_attempts</code> , <code>complete</code> , <code>elapsed_seconds</code> , and <code>updated_at</code> . If the callback produces an error, the function warns once and stops sending updates; the statistical analysis continues.

Details

`HTEAllT()` combines predictions from the propensity score, principal score, and outcome mean models to form the data used for effect estimation. It then estimates one treatment-effect trajectory across all observed times and, when requested, uses bootstrap resampling to calculate confidence intervals.

Value

A `pd_hte_pooled` object containing the jointly estimated trajectory, the analysis times, confidence intervals when $B > 0$, model-checking information, and a summary of successful and failed bootstrap attempts. `time_effect_estimable` indicates whether the data allowed a time effect to be included. Displayed estimates are rounded to three decimal places; `boot_mat` stores the unrounded bootstrap coefficients.

Numerical safeguards

Propensity scores are limited to $[0.01, 0.99]$. Their product with estimated survival probabilities under treatment 1 is limited to $[0.005, 0.995]$ when effects are estimated. These limits prevent division by probabilities very close to zero, but they can affect the estimates and do not demonstrate adequate overlap or validate the causal assumptions. Review the returned model information, especially when few subjects remain at risk.

Treatment coding

The implemented estimator uses treatment 1 as the survival-favorable arm: potential survival satisfies $S^1 \geq S^0$ at cutoff. Its always-survivor principal score is therefore the survival probability under treatment 0. If the survival-favorable arm is coded as 0 in the raw data, recode the raw treatment as 1 - A before mapping and standardizing. To report the original contrast, negate the package estimate and transform an interval $[lower, upper]$ to $[-upper, -lower]$. `Mapping()` does not infer or reverse treatment coding.

See Also

[PDRobust-package](#), [pd_methods](#)

Examples

```
data("BiSample", package = "PDRobust")
map <- Mapping(
  id = "id", time = "time", treatment = "A",
  survival = "S", outcome = "Y",
  baseline_time = 0, cutoff_time = 2,
  covariates = c("X1", "X2", "X4"),
  interest_vars = c("X1", "X2"), y_type = "B"
)
pd_dat <- DataStandard(BiSample, map)
fit <- HTEAllT(
  pd_dat,
  A ~ X1 + X2 + X4,
  S ~ X1 + X2 + X4 + A + time,
  Y ~ X1 + X2 + A,
  B = 0
)
fit$summary
```

HTESepT

Separate estimation of heterogeneous treatment effects by time point

Description

Performs separate analyses of heterogeneous treatment effects at each selected time point.

Usage

```
HTESepT(
  data,
  ps_fo,
  prin_fo,
  out_fo,
  target_time,
  B,
  conf_level = 0.95,
  max_attempts = NULL,
  verbose = TRUE,
  progress_callback = NULL
)
```

Arguments

<code>data</code>	Data prepared by <code>DataStandard()</code> .
<code>ps_fo</code>	propensity score model formula
<code>prin_fo</code>	principal score model formula
<code>out_fo</code>	outcome mean model formula
<code>target_time</code>	A non-empty numeric vector containing timepoints of interest in standardized form. Baseline is allowed.
<code>B</code>	Number of bootstrap replications. Use 0 for point estimates only.
<code>conf_level</code>	The confidence level for Wald intervals calculated from bootstrap standard errors.
<code>max_attempts</code>	The maximum number of resampling attempts allowed to obtain B successful bootstrap replications. Defaults to 10B. Resampling stops once B successful replications are obtained or when the maximum number of attempts is reached, whichever occurs first. Thus, fewer than B successful replications may be returned if the maximum number of attempts is reached.
<code>verbose</code>	If TRUE, print bootstrap progress messages.
<code>progress_callback</code>	An optional function for receiving bootstrap progress updates. It is called before model fitting, after the point estimate, after every bootstrap attempt, and when bootstrapping finishes. Each update is a named list containing stage, successful, requested, attempts, max_attempts, failed_attempts, complete, elapsed_seconds, and updated_at. If the callback produces an error, the function warns once and stops sending updates; the statistical analysis continues.

Details

HTEsepT() combines predictions from the propensity score, principal score, and outcome mean models to form the data used for effect estimation. It fits a separate treatment-effect model at each selected time and, when requested, uses bootstrap resampling to calculate confidence intervals.

Value

A pd_hte_timevarying object containing the estimate for each requested time, confidence intervals when $B > 0$, model-checking information, and a summary of successful and failed bootstrap attempts. Displayed estimates are rounded to three decimal places; boot_mat stores the unrounded bootstrap coefficients.

Numerical safeguards

Propensity scores are limited to $[0.01, 0.99]$. Their product with estimated survival probabilities under treatment 1 is limited to $[0.005, 0.995]$ when effects are estimated. These limits prevent division by probabilities very close to zero, but they can affect the estimates and do not demonstrate adequate overlap or validate the causal assumptions. Review the returned model information, especially when few subjects remain at risk.

Treatment coding

The implemented estimator uses treatment 1 as the survival-favorable arm: potential survival satisfies $S^1 \geq S^0$ at cutoff. Its always-survivor principal score is therefore the survival probability under treatment 0. If the survival-favorable arm is coded as 0 in the raw data, recode the raw treatment as 1 - A before mapping and standardizing. To report the original contrast, negate the package estimate and transform an interval [lower, upper] to [-upper, -lower]. Mapping() does not infer or reverse treatment coding.

See Also

[PDRobust-package](#), [pd_methods](#)

Examples

```
data("BiSample", package = "PDRobust")
map <- Mapping(
  id = "id", time = "time", treatment = "A",
  survival = "S", outcome = "Y",
  baseline_time = 0, cutoff_time = 2,
  covariates = c("X1", "X2", "X4"),
  interest_vars = c("X1", "X2"), y_type = "B"
)
pd_dat <- DataStandard(BiSample, map)
fit <- HTEsepT(
  pd_dat,
  A ~ X1 + X2 + X4,
  S ~ X1 + X2 + X4 + A + time,
  Y ~ X1 + X2 + A,
  target_time = c(0, 2), B = 0
)
```

```
)
fit$summary
```

ImperfectConSample	<i>Imperfect Continuous Longitudinal Example Data</i>
--------------------	---

Description

A deliberately imperfect continuous-outcome longitudinal data set derived from a simulated continuous-outcome panel. The data mimic common issues encountered in raw clinical data exports while remaining recoverable using [DataCheck](#) and [DataStandard](#) with drop = TRUE.

Usage

```
ImperfectConSample
```

Format

A data frame with 599 rows and 11 variables in long format, with one row per recorded subject and visit:

patient_id Noncanonical character subject identifier.
 visit_month Character-encoded visit time in months.
 treatment Character-encoded binary treatment assignment.
 alive_status Character-encoded binary survival or intermediate status.
X1, X2, X3 Continuous baseline covariates.
X4, X5, X6 Binary baseline covariates.
 clinical_outcome Continuous longitudinal clinical outcome.

Details

The data include nonstandard subject identifiers, character-encoded visit times and binary variables, unsorted records, an incomplete longitudinal record, missing required covariate values, a missing outcome among survivors, and a record with a missing subject identifier. Structural outcome missingness for records with alive_status = 0 is retained.

Source

Simulated for package examples.

Examples

```
data("ImperfectConSample", package = "PDRobust")
head(ImperfectConSample)
```

Mapping

*Identify variables and analysis times for PDRobust***Description**

Records which columns contain the subject ID, time, treatment, survival, outcome, and covariates, together with the analysis time range and outcome type. Other package functions use this information to interpret the data consistently.

Usage

```
Mapping(
  id,
  time,
  treatment,
  survival,
  outcome,
  baseline_time,
  cutoff_time,
  covariates,
  interest_vars,
  y_type
)
```

Arguments

<code>id</code>	A single character string naming the subject ID column.
<code>time</code>	A single character string naming the time column.
<code>treatment</code>	A single character string naming the treatment column. For causal estimation, code the survival-favorable arm as 1 and the other arm as 0; see PDRobust-package for the convention and assumptions. The function does not determine which arm is survival-favorable.
<code>survival</code>	A single character string naming the column that records survival or another intermediate status.
<code>outcome</code>	A single character string naming the outcome column.
<code>baseline_time</code>	A single finite number giving the baseline time in the original time scale.
<code>cutoff_time</code>	The time point, on the original time scale, at which the always-survivor principal stratum is defined.
<code>covariates</code>	A character vector naming all variables used as predictors in the propensity score, principal score, or outcome mean models.
<code>interest_vars</code>	A character vector specifying the names of variables used to evaluate heterogeneous treatment effects. Each variable must also be included in <code>covariates</code> .
<code>y_type</code>	The outcome type: "C" for continuous or "B" for binary.

Details

All ten arguments are required. `target_time` is specified separately when calling `HTESepT()` because it selects time points for that analysis rather than describing the data.

Value

A `pd_mapping` object that can be supplied to `DataCheck()` and `DataStandard()`.

Examples

```
map <- Mapping(
  id = "id", time = "time", treatment = "A",
  survival = "S", outcome = "Y",
  baseline_time = 3,
  cutoff_time = 9,
  covariates = c("X1", "X2", "X4"),
  interest_vars = c("X1", "X2"),
  y_type = "C"
)
map
```

ORCI

Estimate covariate associations with survival at the cutoff time

Description

Estimates odds ratios with confidence intervals for associations between covariates and survival at the cutoff time within a selected treatment group.

Usage

```
ORCI(data, formula, a, conf_level = 0.95)
```

Arguments

<code>data</code>	Data prepared by <code>DataStandard()</code> .
<code>formula</code>	A logistic regression formula with the survival variable on the left-hand side and the covariates of interest on the right-hand side.
<code>a</code>	The treatment group to analyze at the cutoff time, either 0 or 1.
<code>conf_level</code>	The confidence level, expressed as a single number between 0 and 1. Defaults to 0.95.

Details

`ORCI()` fits the supplied logistic regression model using observations from treatment group `a` at the cutoff time. It reports an odds ratio and Wald confidence interval for every non-intercept coefficient that can be estimated. Covariates are not selected according to statistical significance.

Value

An odds_ratios object containing odds-ratio estimates and confidence intervals, the fitted model, model-checking information, and a forest plot. Reported estimates are rounded to three decimal places.

Examples

```
data("BiSample", package = "PDRobust")
map <- Mapping(
  id = "id", time = "time", treatment = "A",
  survival = "S", outcome = "Y",
  baseline_time = 0, cutoff_time = 2,
  covariates = c("X1", "X2", "X4"),
  interest_vars = c("X1", "X2"), y_type = "B"
)
pd_dat <- DataStandard(BiSample, map)
result <- ORCI(
  pd_dat, S ~ X1 + X2 + X4, a = 0
)
result$forestplotdat
```

OutPred

Estimate outcome predictions

Description

Fits the outcome mean model and predicts each row of pred_dat under treatment a and survival status 1. The model is fitted again each time the function is called.

Usage

```
OutPred(out_fo, fit_dat, pred_dat, a, mapping, ...)
```

Arguments

out_fo	outcome mean model formula
fit_dat	A data frame containing the observations used to fit the outcome mean model.
pred_dat	A data frame containing the observations for which outcome predictions are requested.
a	The treatment level under which outcomes are predicted, either 0 or 1.
mapping	A pd_mapping object. Its outcome type determines whether the function uses linear regression or logistic regression.
...	Additional arguments passed to stats::lm() or stats::glm().

Value

A numeric vector of predicted outcome means, one for each row of `pred_dat`, rounded to three decimal places.

Examples

```
data("BiSample", package = "PDRobust")
map <- Mapping(
  id = "id", time = "time", treatment = "A",
  survival = "S", outcome = "Y",
  baseline_time = 0, cutoff_time = 2,
  covariates = c("X1", "X2", "X4"),
  interest_vars = c("X1", "X2"), y_type = "B"
)
pd_dat <- DataStandard(BiSample, map)
mu1 <- OutPred(Y ~ X1 + X2 + A + S, pd_dat, pd_dat, a = 1, mapping = map)
head(mu1)
```

PrinPred

Estimate cumulative principal scores

Description

Fits the principal score model and returns each row's estimated probability of surviving from baseline through its observed time under treatment `a`. All observed times from baseline through cutoff are used, and the model is fitted again each time the function is called.

Usage

```
PrinPred(prin_fo, fit_dat, pred_dat, a, mapping, ...)
```

Arguments

<code>prin_fo</code>	principal score model formula
<code>fit_dat</code>	A data frame containing the observations used to fit the model.
<code>pred_dat</code>	A data frame containing the observations for which cumulative survival probabilities are requested.
<code>a</code>	The treatment level under which survival probabilities are predicted, either 0 or 1.
<code>mapping</code>	A <code>pd_mapping</code> object that identifies the variables and analysis times.
<code>...</code>	Additional arguments passed to <code>stats::glm()</code> .

Details

When the data contain multiple times, each post-baseline observation is used to model the next survival step only if the subject was alive at the previous observed time. If the data contain only one observed time, all complete observations at that time are used.

Value

A numeric vector of cumulative survival probabilities, one for each row of `pred_dat`, rounded to three decimal places.

Examples

```
data("BiSample", package = "PDRobust")
map <- Mapping(
  id = "id", time = "time", treatment = "A",
  survival = "S", outcome = "Y",
  baseline_time = 0, cutoff_time = 2,
  covariates = c("X1", "X2", "X4"),
  interest_vars = c("X1", "X2"), y_type = "B"
)
pd_dat <- DataStandard(BiSample, map)
score0 <- PrinPred(
  S ~ X1 + X2 + X4 + A + time,
  pd_dat, pd_dat, a = 0, mapping = map
)
head(score0)
```

PrinSDiag

Evaluate covariate balance for the principal score model

Description

Calculates a standardized balance statistic for each numeric covariate at the cutoff time after accounting for treatment assignment and estimated survival. Values nearer zero indicate better balance between the weighted treatment groups.

Usage

```
PrinSDiag(data, ps_fo, prin_fo)
```

Arguments

<code>data</code>	Data prepared by <code>DataStandard()</code> .
<code>ps_fo</code>	propensity score model formula
<code>prin_fo</code>	principal score model formula

Details

The function fits both the propensity score and principal score models. It uses all observed times from baseline through cutoff to estimate cumulative survival probabilities, limits propensity scores to $[0.01, 0.99]$, and then calculates the balance statistics at the cutoff time.

Value

A PrinSDiag object containing the standardized balance statistics, estimated probabilities, and a diagnostic plot. Balance statistics are rounded to three decimal places.

Examples

```
data("BiSample", package = "PDRobust")
map <- Mapping(
  id = "id", time = "time", treatment = "A",
  survival = "S", outcome = "Y",
  baseline_time = 0, cutoff_time = 2,
  covariates = c("X1", "X2", "X4"),
  interest_vars = c("X1", "X2"), y_type = "B"
)
pd_dat <- DataStandard(BiSample, map)
result <- PrinSDiag(
  pd_dat,
  A ~ X1 + X2 + X4,
  S ~ X1 + X2 + X4 + A + time
)
result$statistics
```

print.pd_mapping	<i>Print, plot, and subset PDRobust results</i>
------------------	---

Description

Provides standard ways to print analysis summaries, draw stored plots, and select rows or columns from data prepared by DataStandard(). Plotting a result does not refit its model.

Usage

```
## S3 method for class 'pd_mapping'
print(x, ...)

## S3 method for class 'pd_data_check'
print(x, ...)

## S3 method for class 'pd_data'
x[...]
```

```
## S3 method for class 'pd_hte_timevarying'
print(x, ...)

## S3 method for class 'pd_hte_pooled'
print(x, ...)
```

```

## S3 method for class 'PSDiag'
print(x, ...)

## S3 method for class 'PrinSDiag'
print(x, ...)

## S3 method for class 'odds_ratios'
print(x, ...)

## S3 method for class 'QR'
print(x, ...)

## S3 method for class 'SA'
print(x, ...)

## S3 method for class 'pd_hte_timevarying'
plot(x, ...)

## S3 method for class 'pd_hte_pooled'
plot(x, ...)

## S3 method for class 'PSDiag'
plot(x, ...)

## S3 method for class 'PrinSDiag'
plot(x, ...)

## S3 method for class 'odds_ratios'
plot(x, ...)

```

Arguments

<code>x</code>	An object returned by a PDRobust function. For <code>[]</code> , this must be a data frame returned by <code>DataStandard()</code> .
<code>...</code>	For subsetting, arguments passed to the next <code>[]</code> method, including row and column indices and drop. For printing and plotting, additional arguments are accepted for generic compatibility but ignored.

Details

Subsetting preserves the stored information but does not check the data again. Before analyzing subsetting or edited data, validate them because removing rows or columns can break the required longitudinal structure. `QR()` has a print method but no package-specific plot method.

Value

Print methods show the main result and invisibly return `x`. When a result contains a plot, printing also draws it; `SA` draws all stored sensitivity plots. Plot methods invisibly return the stored `ggplot`

object. Subsetting returns the selected data and preserves its PDRobust mapping and preparation information when the result remains a data frame.

Examples

```
data("BiSample", package = "PDRobust")
map <- Mapping(
  id = "id", time = "time", treatment = "A", survival = "S", outcome = "Y",
  baseline_time = 0, cutoff_time = 2,
  covariates = c("X1", "X2", "X4"),
  interest_vars = c("X1", "X2"), y_type = "B"
)
print(map)
print(DataCheck(BiSample, map))
prepared <- DataStandard(BiSample, map)
prepared[1:3, ]
diagnostic <- PSDiag(prepared, A ~ X1 + X2 + X4)
print(diagnostic)
p <- plot(diagnostic)
```

PSDiag

Evaluate how well a propensity score model performs

Description

Calculates the standardized mean difference (SMD) for each covariate before and after propensity score weighting.

Usage

```
PSDiag(data, ps_fo)
```

Arguments

data	Data prepared by DataStandard().
ps_fo	propensity score model formula

Details

PSDiag() fits the propensity score model using baseline observations and uses inverse-probability-of-treatment weighting to make the treatment groups more comparable. It limits estimated probabilities to $[0.01, 0.99]$ to avoid extremely large weights. A smaller absolute SMD after weighting indicates better balance for that covariate.

Value

A PSDiag object containing SMDs before and after weighting, the estimated propensity scores and weights, and a balance plot. SMDs are rounded to three decimal places.

Examples

```
data("BiSample", package = "PDRobust")
map <- Mapping(
  id = "id", time = "time", treatment = "A",
  survival = "S", outcome = "Y",
  baseline_time = 0, cutoff_time = 2,
  covariates = c("X1", "X2", "X4"),
  interest_vars = c("X1", "X2"), y_type = "B"
)
pd_dat <- DataStandard(BiSample, map)
result <- PSDiag(pd_dat, A ~ X1 + X2 + X4)
result$smd_after
```

PSPred

*Estimate propensity scores***Description**

Fits a logistic model using baseline observations from `fit_dat` and returns each row's estimated probability of receiving treatment 1 in `pred_dat`. The model is fitted again each time the function is called.

Usage

```
PSPred(ps_fo, fit_dat, pred_dat, mapping, ...)
```

Arguments

<code>ps_fo</code>	propensity score model formula
<code>fit_dat</code>	A data frame containing the baseline observations used to fit the model.
<code>pred_dat</code>	A data frame containing the observations for which propensity scores are requested.
<code>mapping</code>	A <code>pd_mapping</code> object that identifies the treatment and time columns and the baseline time.
<code>...</code>	Additional arguments passed to <code>stats::glm()</code> .

Value

A numeric vector of propensity scores, one for each row of `pred_dat`, rounded to three decimal places.

Examples

```
data("BiSample", package = "PDRobust")
map <- Mapping(
  id = "id", time = "time", treatment = "A",
  survival = "S", outcome = "Y",
  baseline_time = 0, cutoff_time = 2,
  covariates = c("X1", "X2", "X4"),
  interest_vars = c("X1", "X2"), y_type = "B"
)
pd_dat <- DataStandard(BiSample, map)
ps <- PSPred(A ~ X1 + X2 + X4, pd_dat, pd_dat, map)
head(ps)
```

QR	<i>Summary statistics of covariates within the always-survivor principal stratum</i>
----	--

Description

Estimates the user-specified quantile for continuous covariates and the mean of covariates for subjects within the always-survivor principal stratum.

Usage

```
QR(data, prin_fo, quantile_level = 0.5)
```

Arguments

<code>data</code>	Data prepared by <code>DataStandard()</code> .
<code>prin_fo</code>	principal score model formula
<code>quantile_level</code>	One or more quantiles to estimate, expressed as probabilities strictly between 0 and 1. Defaults to the median (0.5).

Details

`QR()` uses estimated survival probabilities under treatment 0 to weight the numeric variables listed in `interest_vars`. It reports a weighted mean for every variable. For variables with more than two observed values, it also reports the requested weighted quantiles. Variables with no more than two observed values are treated as binary and receive a mean but no quantile.

Value

A QR object containing the weighted means, requested quantiles, variable-type indicators, and weights. Reported means and quantiles are rounded to three decimal places.

Examples

```
data("BiSample", package = "PDRobust")
map <- Mapping(
  id = "id", time = "time", treatment = "A",
  survival = "S", outcome = "Y",
  baseline_time = 0, cutoff_time = 2,
  covariates = c("X1", "X2", "X4"),
  interest_vars = c("X1", "X2"), y_type = "B"
)
pd_dat <- DataStandard(BiSample, map)
result <- QR(
  pd_dat,
  S ~ X1 + X2 + X4 + A + time,
  quantile_level = c(0.25, 0.5, 0.75)
)
result$mean
```

SA

Sensitivity analysis of outcome mean model misspecification

Description

Examines how time-specific heterogeneous treatment effect estimates change when random noise is added to the outcome. The amount of noise is determined from the observed outcome variance at each analysis time, and subjects from both treatment groups at the cutoff contribute to effect estimation.

Usage

```
SA(data, ps_fo, prin_fo, out_fo, ratiovec = c(0, 0.05, 0.1))
```

Arguments

data	Continuous- or binary-outcome data prepared by <code>DataStandard()</code> .
ps_fo	propensity score model formula
prin_fo	principal score model formula
out_fo	outcome mean model formula
ratiovec	One or more nonnegative numbers that set the added-noise variance as a proportion of the observed outcome variance. Use 0 for a scenario with no added noise.

Details

For each observed analysis time and each value in `ratiovec`, `SA()` adds mean-zero random noise whose variance equals that value multiplied by the observed outcome variance. It then re-estimates the heterogeneous treatment effect for that time.

For continuous outcomes, the perturbed outcomes are used both to refit the outcome mean model and to estimate the treatment effect. For binary outcomes, the outcome mean model is fitted to the original binary outcomes, while the perturbed outcomes are used only during effect estimation. Binary treatment effects use the same bounded scale as `HTESepT()`.

The three nuisance models are fitted again as needed for each scenario. The results show sensitivity to this particular form of random outcome noise; they do not cover every possible model error or violation of the causal assumptions. Set an R random seed before calling `SA()` to reproduce the same noise.

Value

An SA object containing estimates for every analysis time and noise level, model-checking information, warnings, and plots. Displayed estimates are rounded to three decimal places.

Examples

```
data("BiSample", package = "PDRobust")
map <- Mapping(
  id = "id", time = "time", treatment = "A",
  survival = "S", outcome = "Y",
  baseline_time = 0, cutoff_time = 2,
  covariates = c("X1", "X2", "X4"),
  interest_vars = c("X1", "X2"), y_type = "B"
)
pd_dat <- DataStandard(BiSample, map)
set.seed(20260912)
result <- SA(
  pd_dat,
  A ~ X1 + X2 + X4,
  S ~ X1 + X2 + X4 + A + time,
  Y ~ X1 + X2 + A,
  ratiovec = c(0, 0.05)
)
head(result$data)
```

Index

* datasets

- BiSample, [2](#)
- ImperfectConSample, [10](#)
- `[.pd_data (print.pd_mapping), 16`
- BiSample, [2](#)
- DataCheck, [3](#), [10](#)
- DataStandard, [4](#), [10](#)
- HTEAllT, [5](#)
- HTESepT, [8](#)
- ImperfectConSample, [10](#)
- Mapping, [11](#)
- ORCI, [12](#)
- OutPred, [13](#)
- pd_methods, [5](#), [7](#), [9](#)
- pd_methods (print.pd_mapping), [16](#)
- PDRobust-package, [7](#), [9](#), [11](#)
- plot.odds_ratios (print.pd_mapping), [16](#)
- plot.pd_hte_pooled (print.pd_mapping),
[16](#)
- plot.pd_hte_timevarying
(print.pd_mapping), [16](#)
- plot.PrinSDiag (print.pd_mapping), [16](#)
- plot.PSDiag (print.pd_mapping), [16](#)
- PrinPred, [14](#)
- PrinSDiag, [15](#)
- print.odds_ratios (print.pd_mapping), [16](#)
- print.pd_data_check (print.pd_mapping),
[16](#)
- print.pd_hte_pooled (print.pd_mapping),
[16](#)
- print.pd_hte_timevarying
(print.pd_mapping), [16](#)
- print.pd_mapping, [16](#)
- print.PrinSDiag (print.pd_mapping), [16](#)

- print.PSDiag (print.pd_mapping), [16](#)
- print.QR (print.pd_mapping), [16](#)
- print.SA (print.pd_mapping), [16](#)
- PSDiag, [18](#)
- PSPred, [19](#)
- QR, [20](#)
- SA, [21](#)